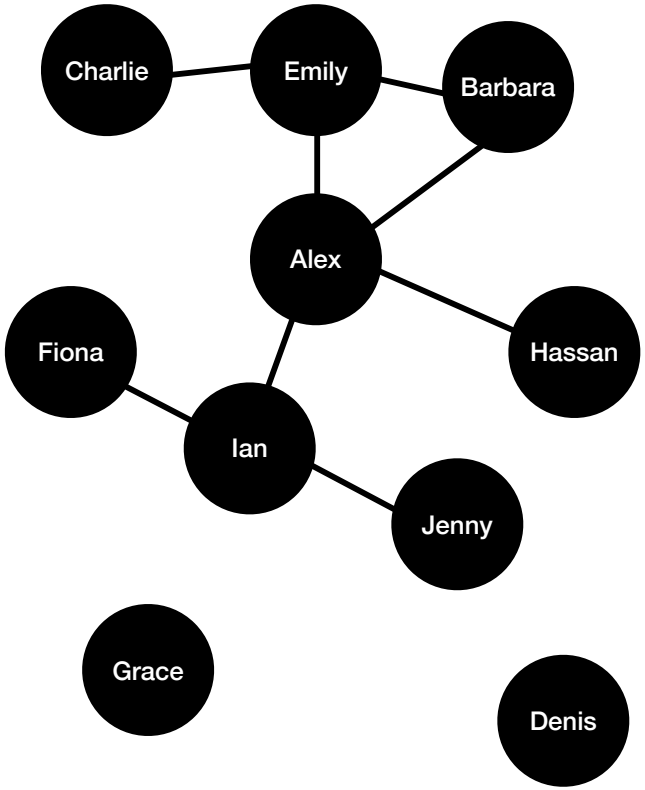
CS430/910: Foundations of Data Analytics

Social Network Analysis 2 | Ayse Sunar

Clustering Coefficient

- Most social networks look different to “random graphs”
- Much more likely to see a tie between a pair of neighbouring nodes than by chance
  - These are called triangles (i.e. two of my friends are also friends)
- Applies to undirected graphs, but there have been variations created which can apply to directed graphs.

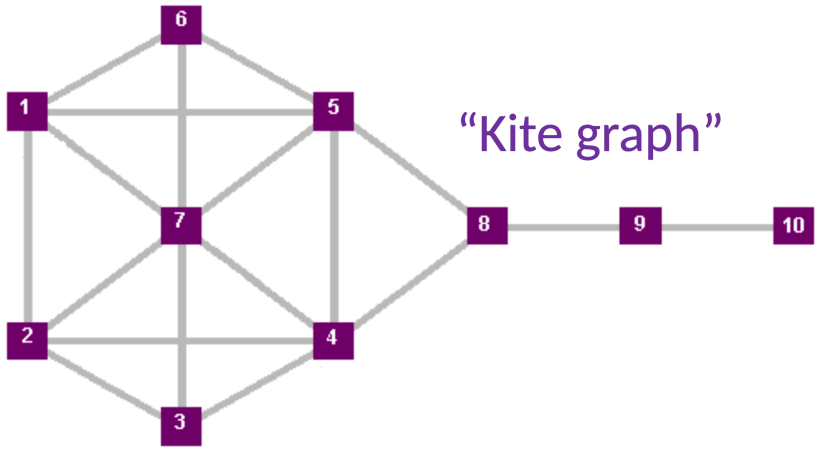


Finding Important Nodes

- Much work on finding important or influential nodes
  - Degree: find node with highest degree
  - Not always meaningful: can engineer nodes with high degree
  - E.g. could plant nodes which all follow/friend each other
- Other more complicated measures can be used
  - E.g. betweenness centrality.

Betweenness & Eccentricity

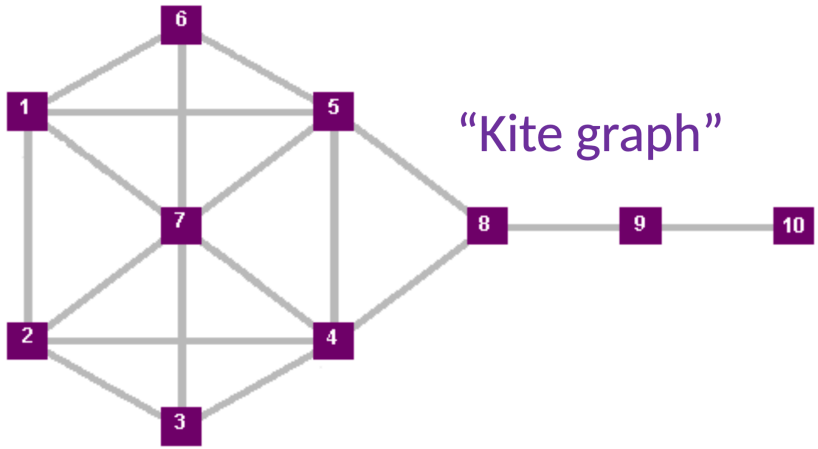
- *Betweenness*: fraction of shortest paths passing through a node
  - In the kite graph, #8 is on many shortest paths
- The *eccentricity* of a node is the maximum distance to any other node in the graph
  - Eccentricity of #5 is 3 (realised by (5,10))
- If we calculate the eccentricity for all nodes:
  - The **minimum** eccentricity is known as the *radius*.
  - The **maximum** eccentricity is known as the *diameter*.



"Kite graph"

Eccentricity

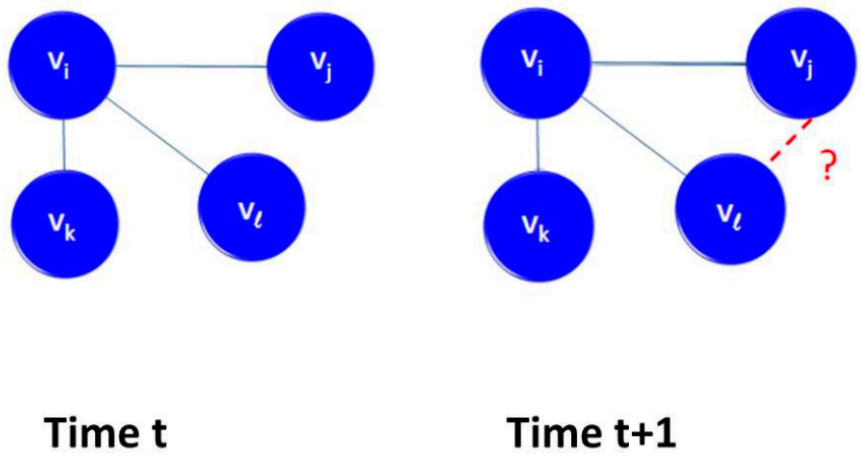
- A *central node* is one that achieves the minimum eccentricity
  - #8 is the only central node here, with an eccentricity of 2



"Kite graph"

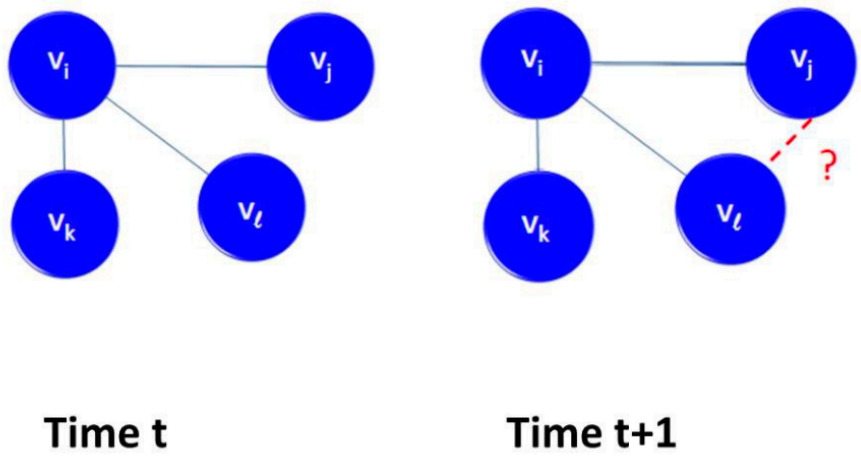
Tie Prediction on Graphs

- A basic problem: how can we predict which new ties will be formed?
- Can try to model this as classification:
  - **Input**: features of each node, and ties
  - **Predict**: for a given pair of nodes, will a tie be formed?
  - In a social network, features are typically demographics and interests



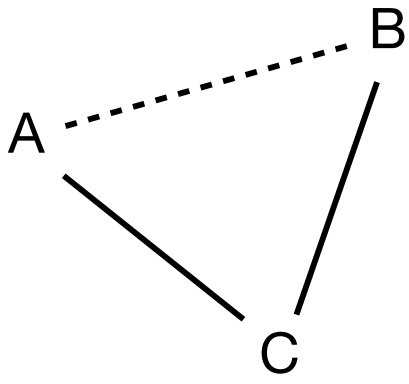
Tie Prediction on Graphs

- Tie prediction is used e.g. for recommending friends to add
- Directly using a classifier to predict ignores the graph structure:
  - People are unlikely to add friends just based on interests etc.
  - Instead, try to use proximity in the graph to predict ties



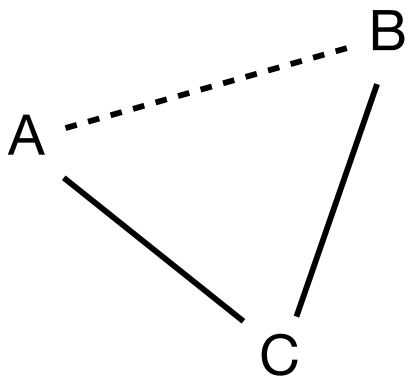
Common Neighbours

- Simplest model: predict that common neighbours will form a tie
  - If (C, A) and (C, B) are ties, predict (A, B)
  - Let $N(C)$ be the neighbours of C, predict $\{(A, B) : A \in N(C), B \in N(C)\}$
  - Plausible: mutual friends, similar interests...



Common Neighbours

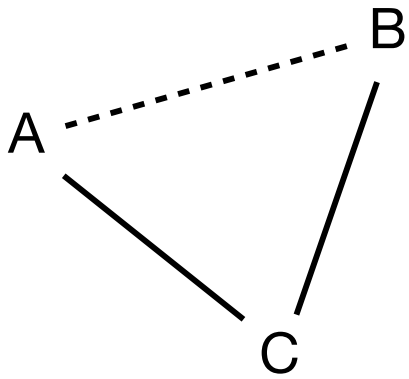
- Problem: This may predict a lot of ties!
  - On Facebook, users have on average 338[1] friends
  - This therefore would predict thousands of ties for each user!
- Instead, give a score to each A, B pair. Then, predict the highest scoring pairs.



[1]https://www.pewresearch.org/fact-tank/2014/02/03/what-people-like-dislike-about-facebook/

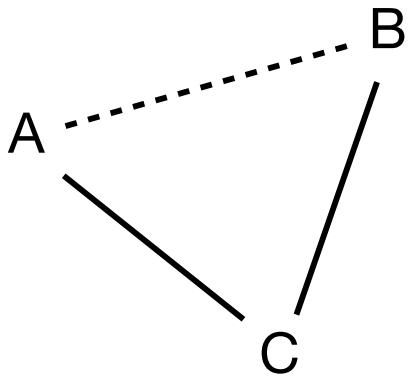
Weighting Common Neighbours

- Weight the prediction by the number of common neighbours:
  - If A and B have 10 mutual friends, they're more likely to connect than if they have 1
  - score$(A, B) = |N(A) \cap N(B)|$
- *Jaccard Coefficient*: what fraction of neighbours that are common:
  - Count common neighbours of A and B, divided by total neighbours
  - score$(A, B) = \dfrac{|N(A) \cap N(B)|}{|N(A) \cup N(B)|}$



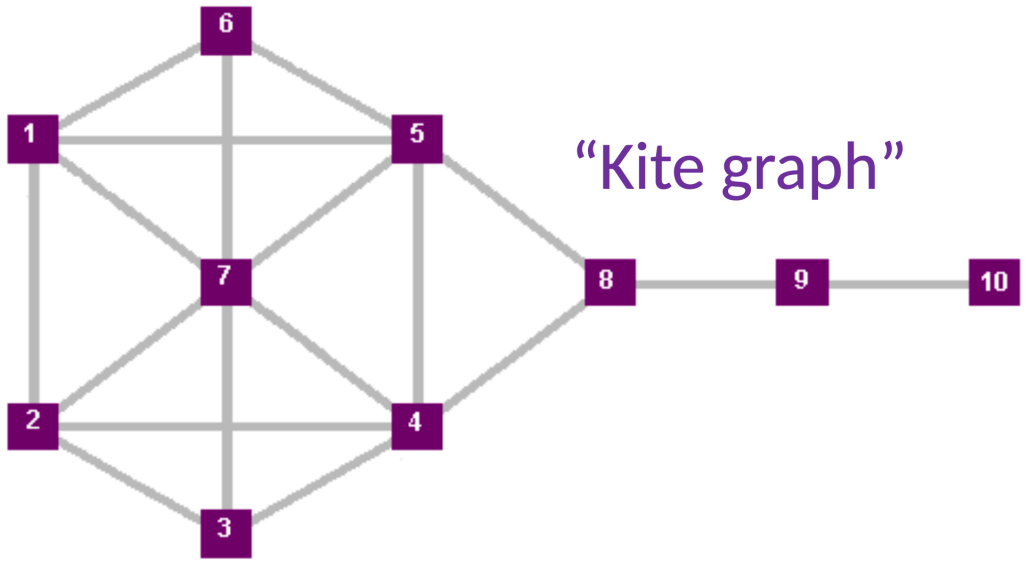
Weighting Common Neighbours

- Adamic-Adar: weight common neighbours by their degree
  - Sum over common neighbours, 1/log(number of neighbours)
  - score$(A, B) = \sum_{C \in N(A) \cap N(B)} \frac{1}{log(|N(C)|)}$
- Preferential attachment: high-degree nodes more likely to form a tie
  - score$(A, B) = |N(A)| * |N(B)|$
- Survey: <u>The Link Prediction Problem for Social Networks</u>



Measures Based on Random Walks

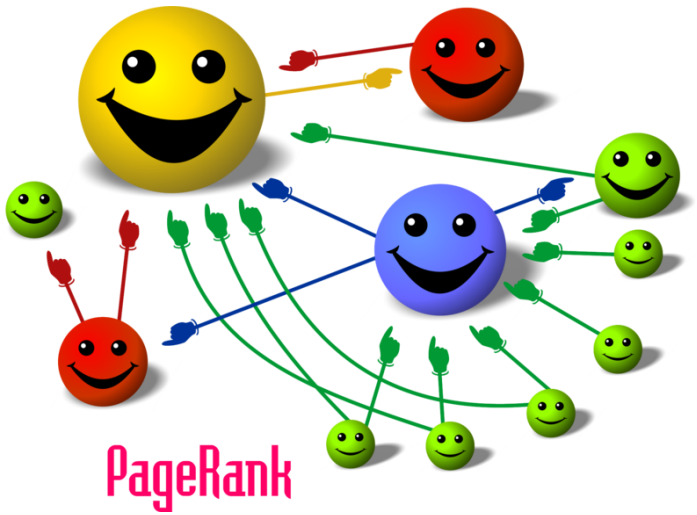
- If I take a random walk through the network, which nodes am I more likely to end up at?
- Many measures of importance based on random walks
  - PageRank (ranking web pages) is interpreted as a random walk
- Random walks on kite graph likely to end up at (7), less likely to end up at (10)



"Kite graph"

PageRank

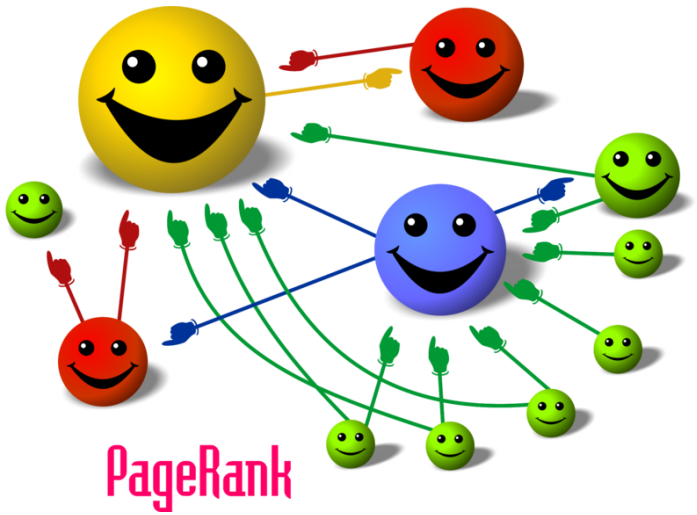
- Given a (directed) graph, want to compute importance of nodes
  - Most useful web pages on the internet
  - Most significant people in a social network
  - Most influential papers in scientific research literature
- Simple approach: find nodes with highest degree
- Plausible? Has the most nodes pointing to it
- On the web, highest in-degree page used to be adobe.com
  - Get Flash/Acrobat Reader
- Easy to “game”: make lots of webpages that point to mine



PageRank

Towards PageRank (Brin & Page, 1998)

- PageRank algorithm uses more complex properties of graph
  - Originally introduced to rank search results on Google
- Basic idea: Each directed tie in the graph represents a “vote” from the source of the directed tie that the destination is “good"
  - So score a page (node) by the votes it receives.
- Why is this not the same as computing the in-degree?
  1. Weight of the “vote” depends on the score of the page making the vote
  2. Divide the score of a page evenly by the number of outward ties: the more votes made, the lower the weight of each vote



Page, L., Brin, S., Motwani, R. and Winograd, T., 1999. *The PageRank citation ranking: Bringing order to the web*. Stanford InfoLab.

Simple PageRank

- Let's look at a simple version of the algorithm (Simple PageRank).
- The PageRank of a page P_i at iteration $k+1$, denoted $r_{k+1}(P_i)$, is calculated as follows:

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

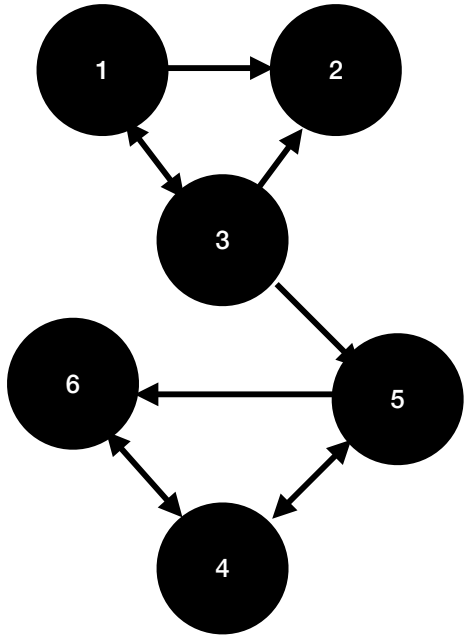
- Where B_{P_i} is the set of pages pointing (with inwards ties) into P_i, and $|P_j|$ is the number of outward ties from page P_j.

Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- $r_0 = \frac{1}{n}$ for all pages P_i, where n is the number of pages.

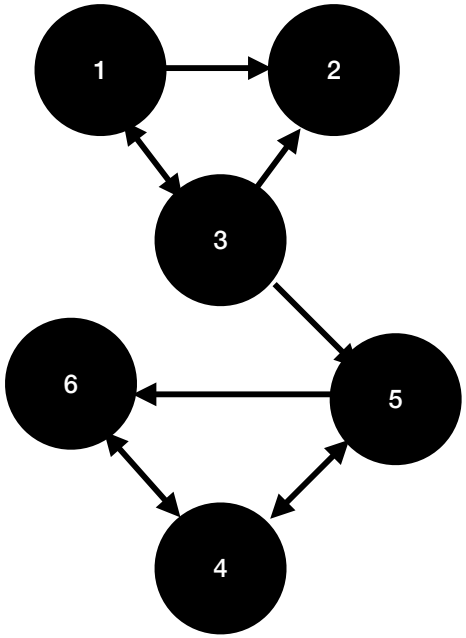
	Iteration 0	Iteration 1	Iteration 2
1	1/6		
2	1/6		
3	1/6		
4	1/6		
5	1/6		
6	1/6		



Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- $B_{P_1} = \{P_3\}$
- $r_1(P_1) = \sum_{P_j \in \{P_3\}} \frac{r_0(P_j)}{|P_j|} = \frac{\frac{1}{6}}{3} = \frac{1}{18}$

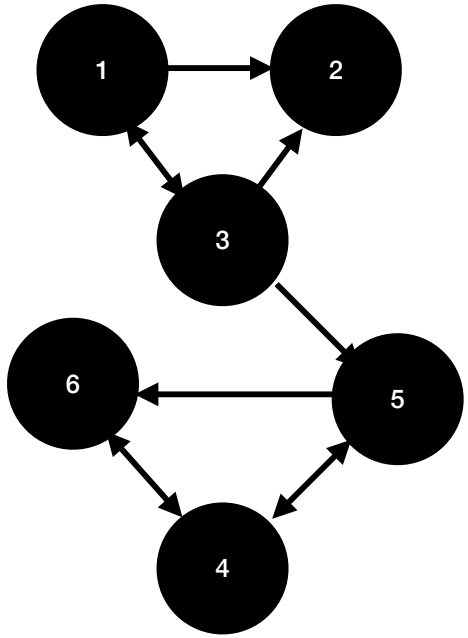


	Iteration 0	Iteration 1	Iteration 2
1	1/6	1/18	
2	1/6		
3	1/6		
4	1/6		
5	1/6		
6	1/6		

Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- $B_{P_2} = \{P_1, P_3\}$
- $r_1(P_2) = \sum_{P_j \in \{P_1, P_3\}} \frac{r_0(P_j)}{|P_j|} = \frac{\frac{1}{6}}{2} + \frac{\frac{1}{6}}{3} = \frac{5}{36}$



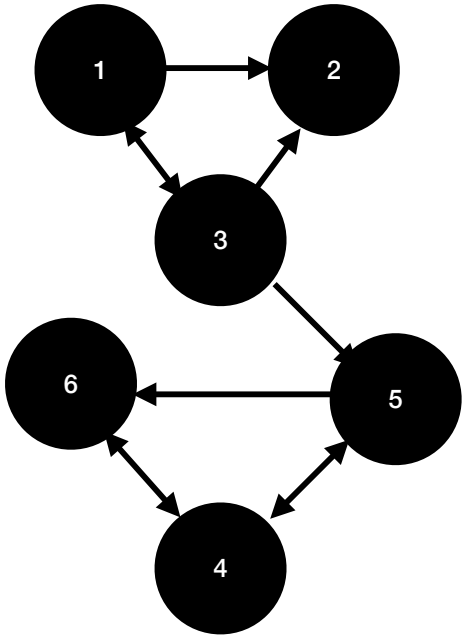
	Iteration 0	Iteration 1	Iteration 2
1	1/6	1/18	
2	1/6	5/36	
3	1/6		
4	1/6		
5	1/6		
6	1/6		

Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- Remainder of Iteration 1 calculated…

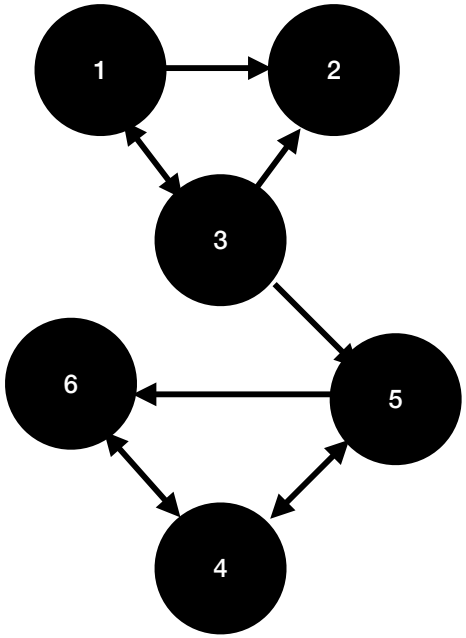
	Iteration 0	Iteration 1	Iteration 2
1	1/6	1/18	
2	1/6	5/36	
3	1/6	1/12	
4	1/6	1/4	
5	1/6	5/36	
6	1/6	1/6	



Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- $B_{P_1} = \{P_3\}$
- $r_2(P_1) = \sum_{P_j \in \{P_3\}} \frac{r_1(P_j)}{|P_j|} = \frac{\frac{1}{12}}{3} = \frac{1}{36}$

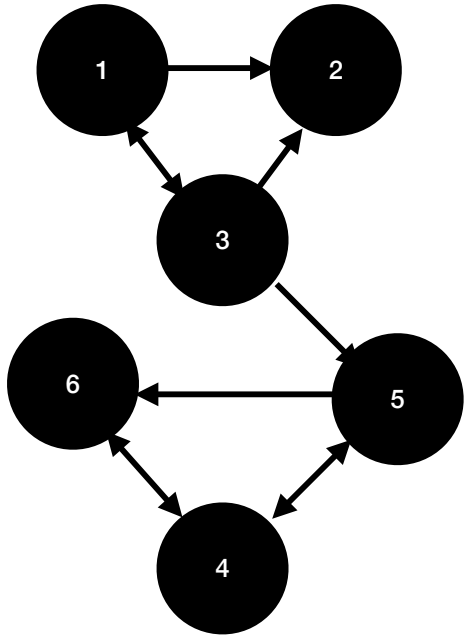


	Iteration 0	Iteration 1	Iteration 2
1	1/6	1/18	1/36
2	1/6	5/36	
3	1/6	1/12	
4	1/6	1/4	
5	1/6	5/36	
6	1/6	1/6	

Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- $B_{P_2} = \{P_1, P_3\}$
- $r_2(P_2) = \sum_{P_j \in \{P_1, P_3\}} \frac{r_1(P_j)}{|P_j|} = \frac{\frac{1}{18}}{2} + \frac{\frac{1}{12}}{3} = \frac{1}{18}$



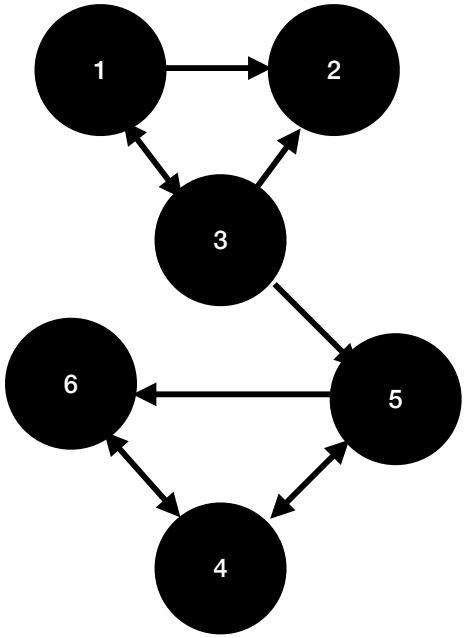
	Iteration 0	Iteration 1	Iteration 2
1	1/6	1/18	1/36
2	1/6	5/36	1/18
3	1/6	1/12	
4	1/6	1/4	
5	1/6	5/36	
6	1/6	1/6	

Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- Remainder of Iteration 2 calculated…

	Iteration 0	Iteration 1	Iteration 2
1	1/6	1/18	1/36
2	1/6	5/36	1/18
3	1/6	1/12	1/36
4	1/6	1/4	17/72
5	1/6	5/36	11/72
6	1/6	1/6	14/72



Simple PageRank

Determining the Rank

- After several iterations, the ranks will eventually converge to stable values.
- For demonstration purposes, assume we've converged after iteration 2.
- Higher value at Iteration 2 = Higher rank.

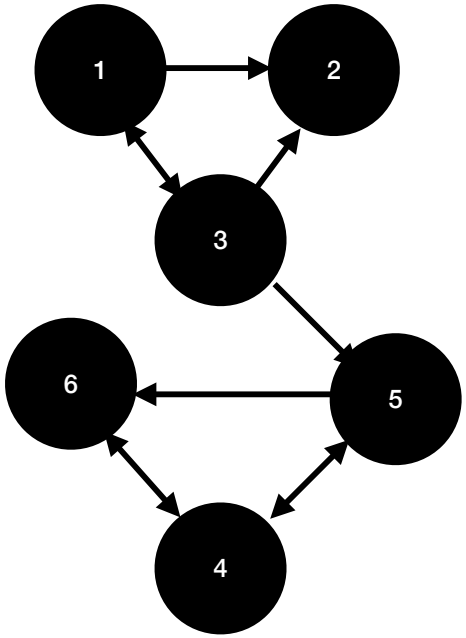
	Iteration 0	Iteration 1	Iteration 2	Rank at Iteration 2
1	1/6	1/18	1/36	5
2	1/6	5/36	1/18	4
3	1/6	1/12	1/36	5
4	1/6	1/4	17/72	1
5	1/6	5/36	11/72	3
6	1/6	1/6	14/72	2

PageRank

Justification

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- Let's say #3 is my page. It came joint last in our rankings previously… I want to boost the page up Google's rankings!

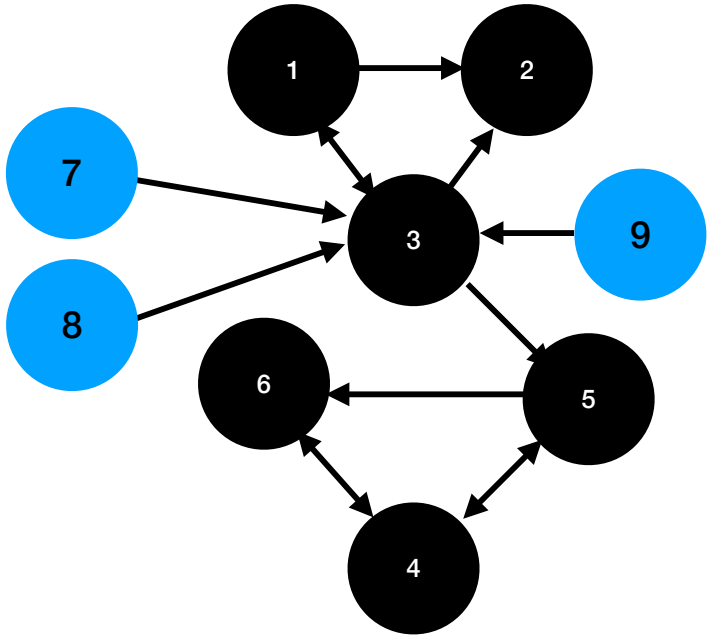


PageRank

Justification

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- Let's say #3 is my page. It came joint last in our rankings previously… I want to boost the page up Google's rankings!
- We've made three new pages, #7, #8 and #9, all which have a directed tie to #3.
- Would it boost the ranking of #3?

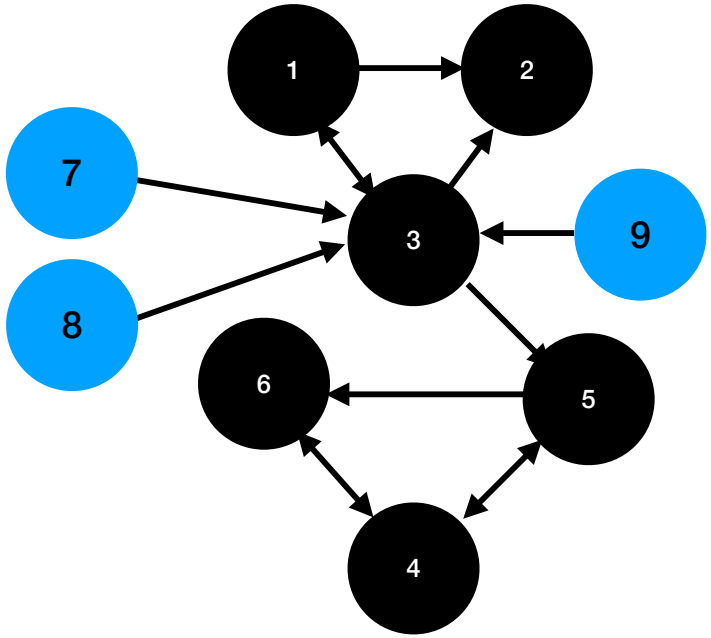


PageRank

Justification

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- Let's say #3 is my page. It came joint last in our rankings previously… I want to boost the page up Google's rankings!
- We've made three new pages, #7, #8 and #9, all which have a directed tie to #3.
- Recall the PageRank equation… B_{P_i} is the set of all pages pointing into page P_i… but no pages are pointing into pages #7, #8 and #9, so their vote is worthless!
- Prevents people artificially boosting their pages by creating more pages.



Simple PageRank

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|}$$

- The summation method ($\sum$) is tedious - computes PageRank one page at a time.
- We can replace the summation method with matrix multiplication.
- Use a single $1 \times n$ row vector, $\boldsymbol{\pi}^T$, to hold the PageRank values for all pages.
- We need an $n \times n$ matrix $\mathbf{H}$, where $\mathbf{H}_{ij} = \dfrac{1}{|P_i|}$ if there is a link from node i to node j.

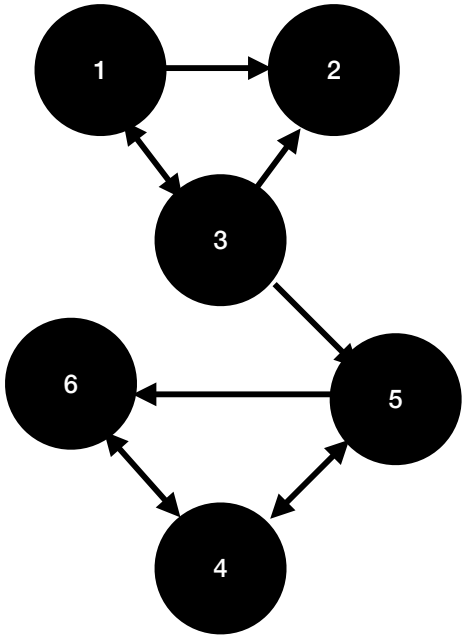
Simple PageRank

- $\boldsymbol{\pi}^{(k)T}$ is the PageRank vector at the kth iteration.
- Using this matrix notation, we can calculate PageRank as follows:

$$\boldsymbol{\pi}^{(k+1)T} = \boldsymbol{\pi}^{(k)T}\mathbf{H}$$

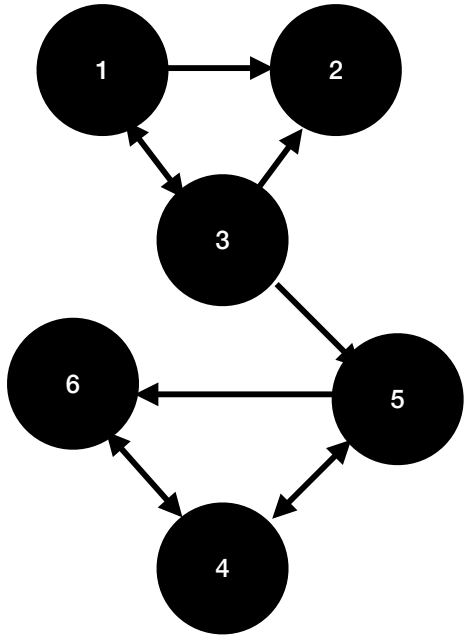
Simple PageRank

- Going back to our graph:
- $$\mathbf{H} = \begin{array}{c} \\ P_1 \\ P_2 \\ P_3 \\ P_4 \\ P_5 \\ P_6 \end{array} \begin{array}{c} \begin{array}{cccccc} P_1 & P_2 & P_3 & P_4 & P_5 & P_6 \end{array} \\ \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1/3 & 0 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \end{array}$$
- $$\boldsymbol{\pi}^{(0)T} = (1/6 \quad 1/6 \quad 1/6 \quad 1/6 \quad 1/6 \quad 1/6)$$



Simple PageRank

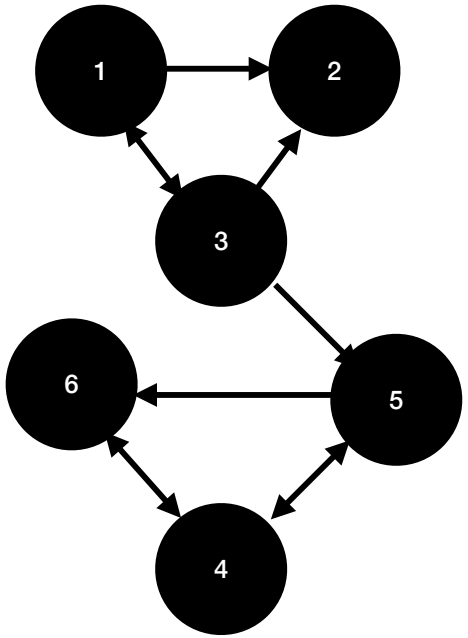
- Going back to our graph:
- $$\boldsymbol{\pi}^{(1)T} = \boldsymbol{\pi}^{(0)T}\mathbf{H}$$
- $$\boldsymbol{\pi}^{(1)T} = (1/6 \quad 1/6 \quad 1/6 \quad 1/6 \quad 1/6 \quad 1/6)\begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1/3 & 0 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$
- $$\boldsymbol{\pi}^{(1)T} = (1/18 \quad 5/36 \quad 1/12 \quad 1/4 \quad 5/36 \quad 1/6)$$



PageRank

An Interpretation

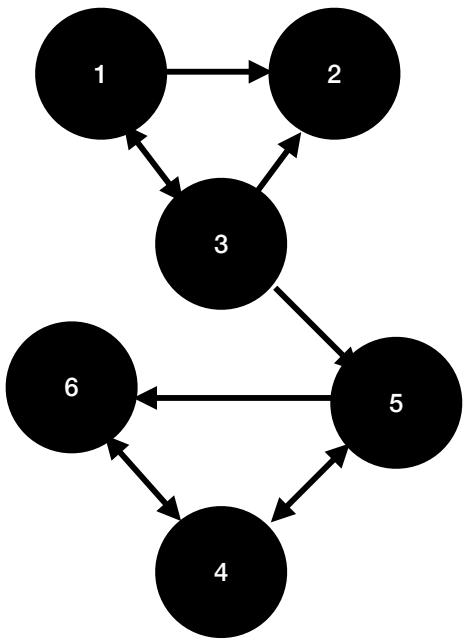
- Consider someone is browsing the web.
- They begin with a random page (node).
  - Say, node 5.
- Node 5 has two outward ties, so they can end up on node 4 or node 6.
  - Probability of ending up on node 4 or 6 is 50% each.
- However, node 3 has 3 outward ties:
  - Probability of ending up on node 1, 2 or 5 is 33.3% each.



PageRank

The Random Surfer Model

- Define a new matrix $\mathbf{S}$, which is equivalent to $\mathbf{H}$, but in the case of any pages with no outward ties, replace row with $1/n$.
- $$\mathbf{S} = \begin{array}{c} \\ P_1 \\ P_2 \\ P_3 \\ P_4 \\ P_5 \\ P_6 \end{array} \begin{array}{c} \begin{array}{cccccc} P_1 & P_2 & P_3 & P_4 & P_5 & P_6 \end{array} \\ \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 1/3 & 1/3 & 0 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \end{array}$$
- This can be achieved by $\mathbf{S} = \mathbf{H} + \mathbf{a}(\frac{1}{n}\mathbf{e}^T)$, where $a_i = 1$ if page i has no outward ties, and $\mathbf{e}^T$ is a row vector of 1s.



PageRank

The Random Surfer Model

- However, people don't keep clicking on web links forever!
  - Starting from a random page
  - With probability α, pick an outgoing tie uniformly from the current page, and follow it
  - With probability $1 - \alpha$, jump to a randomly chosen node/page
  - α is known as the *damping factor*.

α is typically around 0.85.

PageRank

The Random Surfer Model

- To model this mathematically:
  - This updated PageRank is given by:
    - $$\mathbf{G} = \alpha\mathbf{S} + (1-\alpha)\frac{1}{n}\mathbf{e}\mathbf{e}^T$$

    where α is between 0 and 1.
  - $\mathbf{G}$ is called the *Google matrix*.
  - Google's adjusted PageRank method is:

$$\boldsymbol{\pi}^{(k+1)T} = \boldsymbol{\pi}^{(k)T}\mathbf{G}$$

α is typically around 0.85.

PageRank

The Random Surfer Model

- To model this mathematically:
  - This updated PageRank is given by:
    - $$\mathbf{G} = \alpha\mathbf{S} + (1-\alpha)\frac{1}{n}\mathbf{e}\mathbf{e}^T$$

    where α is between 0 and 1.
  - $\mathbf{G}$ is called the *Google matrix*.
  - Google's adjusted PageRank method is:

$$\boldsymbol{\pi}^{(k+1)T} = \boxed{\boldsymbol{\pi}^{(k)T}}\mathbf{G}$$

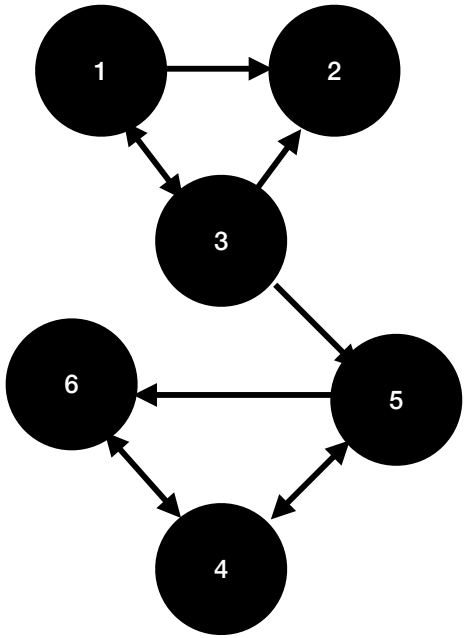
Obtained by finding dominant eigenvector of G.

α is typically around 0.85.

PageRank

Final Points

- Basic PageRank methods struggle with a Cold-Start problem.
  - New pages have very few incoming ties, so have a low ranking.
  - As they have a low ranking, other pages cannot find them to add ties to the new pages!
- Variations of PageRank have been proposed, e.g. Personalised PageRank, SimRank etc. (see the literature for more information).



Other Mining in Social Networks

- Many other data analysis problems studied in social networks
  - Attach strength to ties
    - Not all “friends” are equal: which are the ones that matter?
  - Finding meaningful “groups” in social networks
    - Like clustering, but much more overlap among groups
  - Picking a collection of nodes to maximise “influence”
    - Send free samples of products, gossip/news on movies